

CineSeat Backend

# Backend'imizi Nasıl Kurduk?

CQRS · MediatR · Onion Mimarisi · Auth · Rezervasyon

Bu rehber, CineSeat projesinin backend'inde yaptığımız her şeyi sıfırdan anlatır. Öncelik mimari ve CQRS/MediatR kısmında — kod kod, satır satır, "bu satır ne işe yarıyor" diye açıklanmış. Arkasından Auth, güvenlik, rezervasyon gibi diğer eklemeleri aynı ayrıntıda bulacaksınız. Hiç backend bilmeyen biri okuyup anlayabilsin diye yazıldı.

# İçindekiler

---

## BÖLÜM 1 — Mimarinin Temeli

---

- 1.1 Neden 5 ayrı proje var?
  - 1.2 Bağımlılık yönü kuralı (Onion)
- 

## BÖLÜM 2 — CQRS ve MediatR (öncelikli bölüm)

---

- 2.1 CQRS nedir?
  - 2.2 MediatR hangi sorunu çözer?
  - 2.3 Bir "dilim"in anatomisi (Command/Query/Handler/Validator)
  - 2.4 MediatR nasıl devreye giriyor — DependencyInjection.cs
  - 2.5 Pipeline Behavior — isteğin geçtiği "borular"
  - 2.6 Bir isteğin A'dan Z'ye tam yolculuğu
  - 2.7 Repository Pattern — Application neden EF Core'u tanımıyor
  - 2.8 Hata yönetimi — tek try/catch mantığı
- 

## BÖLÜM 3 — Diğer Backend Eklemeleri

---

- 3.1 Kimlik doğrulama (Auth) ve JWT
  - 3.2 Yetkilendirme (RBAC) — kim neyi yapabilir
  - 3.3 IDOR güvenlik açıkları ve kapatılması
  - 3.4 Katalog modülü (Film, Tür, Kampanya)
  - 3.5 Sosyal modül (Yorum, Favori, Profil)
  - 3.6 Konum & Mekan modülü (Şehir → Koltuk)
  - 3.7 Rezervasyon akışı — en karmaşık parça
  - 3.8 Sayfalama (Pagination)
  - 3.9 Otomatik kurulum (Migrate + Seed)
- 

## BÖLÜM 4 — Şu Anki Durum: Sağlık Kontrolü

---

# Bölüm 1 — Mimarinin Temeli

Detaya girmeden önce: kod neden bu şekilde 5 klasöre/projeye bölünmüş?

## 1.1 Neden 5 ayrı proje var?

Backend'imiz tek bir dosya yığını değil, birbirine bağımlı **5 ayrı C# projesinden** oluşuyor:

```
backend/
├─ Core/
│  ├─ CineSeat.Domain          → Entity'ler (User, Movie, Reservation ...) ve Enum'lar
│  └─ CineSeat.Application      → CQRS burada yaşıyor (Bölüm 2'nin konusu)
├─ Infrastructure/
│  ├─ CineSeat.Infrastructure    → JWT üretimi, şifre hash'leme (dış servisler)
│  └─ CineSeat.Persistence      → Veritabanı bağlantısı, EF Core, migration'lar
└─ Presentation/
   └─ CineSeat.WebAPI           → HTTP isteklerini karşılayan Controller'lar
```

**Benzetme:** Bir restoran düşünün. **Domain** menüdeki yemeklerin tarifleri (ne malzeme kullanılır — asla değişmez). **Application** aşçının yaptığı işler ("müşteri sipariş verdi, yemeği hazırla"). **Persistence** mutfağın kendisi (dolaplar, ocaklar — somut donanım). **WebAPI** garson (müşteriden siparişi alır, mutfağa iletir, tabağı geri getirir). Aşçı (Application), hangi marka ocak kullanıldığını bilmek **zorunda değildir** — sadece "pişir" der.

## 1.2 Bağımlılık yönü kuralı (Onion Mimarisi)

Her proje sadece **belirli** diğer projelere referans verebilir. Bu, her projenin **.csproj** dosyasında **<ProjectReference>** satırlarıyla zorlanır — birisi kuralı yanlışlıkla çiğnemeye çalışırsa **proje derlenmez**.

```
WebAPI —depends on→ Application —depends on→ Domain
└──depends on→ Persistence —depends on→ Application, Domain
                        Infrastructure —depends on→ Application
```

**Altın kural:** Ok her zaman *içeri*, yani **Domain** 'e doğru akar. **Domain** hiçbir şeye bağımlı değildir — kimseyi tanımaz. **Application** ise sadece **Domain** 'i tanır; **Entity Framework'ü, veritabanını, HTTP'yi bilmez**. Bu kuralın somut sonucunu Bölüm 2.7'de (Repository Pattern) göreceksiniz.

## Bölüm 2 — CQRS ve MediatR

öncelikli bölüm

Backend'in kalbi burası. Her özellik (film ekleme, rezervasyon yapma, yorum silme...) bu kalıba göre yazıldı — toplamda 90'dan fazla dosya bu yapıyı kullanıyor.

### 2.1 CQRS nedir?

**CQRS = Command Query Responsibility Segregation** (Komut-Sorgu Sorumluluk Ayrımı). Kulağa karmaşık geliyor ama fikir çok basit:

- **Command (Komut):** Bir şeyi *değiştiren* işlem. "Film ekle", "Rezervasyon yap", "Yorum sil".
- **Query (Sorgu):** Bir şeyi *okuyan* ama *değiştirmeyen* işlem. "Filmleri listele", "Rezervasyonumu getir".

Neden ayırıyoruz? Çünkü okuma ve yazmanın **ihtiyaçları farklı**. Yazarken doğrulama (validation) yapmanız gerekir ("film adı boş olamaz"); okurken buna gerek yoktur ama filtreleme, sıralama, sayfalama gibi başka şeylere ihtiyacınız olur. İkisini aynı fonksiyonda karıştırmak kodu karmaşılaştırır — CQRS bu ikisini **baştan ayrı tutar**.

**Benzetme:** Bir kütüphane düşünün. **Kitap ödünç almak** (Command) — kütüphaneci kaydınızı günceller, kitabın durumunu değiştirir. **Kitap aramak** (Query) — hiçbir şeyi *değiştirmezsiniz*, sadece rafa bakarsınız. İki işlem tamamen farklı butonlar/masalar kullanır — CQRS de kodda aynı ayrımı yapar.

### 2.2 MediatR hangi sorunu çözer?

MediatR olmadan, bir Controller (HTTP isteğini karşılayan sınıf) doğrudan iş mantığını içinde barındırırdı:

**MediatR OLMADAN (kötü örnek — bizde böyle değil):**

```
public class CitiesController
{
    [HttpPost]
    public async Task Create(string cityName)
    {
        // Doğrulama burada, DB erişimi burada, iş kuralı burada...
        if (string.IsNullOrEmpty(cityName)) return BadRequest();
        var city = new City { CityName = cityName };
        _dbContext.Cities.Add(city); // Controller, EF Core'u DOĞRUDAN biliyor
        await _dbContext.SaveChangesAsync();
        return Ok(city.Id);
    }
}
```

Bu yaklaşımın sorunu: Controller'lar zamanla **devasa** olur, her yerde tekrar eden doğrulama kodu birikir, ve Controller (WebAPI katmanı) veritabanını (Persistence katmanı) doğrudan tanımak zorunda kalır — Bölüm 1.2'deki kuralı çiğner.

**MediatR** araya bir "posta kutusu" koyar. Controller artık ne yapılacağını *bilmez*, sadece "şu paketi gönder" der:

**MediatR İLE (bizim gerçek kodumuz):**

```
public class CitiesController : ControllerBase
{
    private readonly IMediator _mediator;

    [HttpPost]
    public async Task<IActionResult> Create([FromBody] CreateCityCommand command)
        => Ok(await _mediator.Send(command)); // tek satır!
}
```

Controller artık sadece 1 satır: `_mediator.Send(command)`. "Bu komutu kim işleyecek, nasıl doğrulanacak, veritabanına nasıl yazılacak" — bunların HİÇBİRİNİ Controller bilmiyor. MediatR, komutun tipine bakıp (`CreateCityCommand`) doğru "işleyiciyi" (Handler) otomatik bulup çalıştırıyor.

## 2.3 Bir "dilim"in anatomisi

Her özellik (Feature), MediatR dünyasında **4 parçadan** oluşan bir "dikey dilim" (vertical slice) olarak yazılır. Örnek: "yeni şehir ekleme" özelliği.

```
Features/Cities/Commands/CreateCity/
├ CreateCityCommand.cs           ← "Ne isteniyor?" (sadece veri taşır)
├ CreateCityCommandHandler.cs    ← "Nasıl yapılır?" (asıl iş mantığı burada)
├ CreateCityCommandValidator.cs  ← "Geçerli mi?" (Handler'dan ÖNCE otomatik çalışır)
└ (varsa) CityDto.cs            ← "Dışarı ne dönülür?"
```

### ① Command — "Ne isteniyor?"

```
Features/Cities/Commands/CreateCity/CreateCityCommand.cs

public class CreateCityCommand : IRequest<long>
{
    public string CityName { get; set; } = string.Empty;
}
```

Sadece bir **veri taşıyıcı**. `IRequest<long>` demek: "Bu komut işlendiğinde geriye bir `long` (yeni şehrin id'si) dönecek" demektir. Hiç iş mantığı yok — sadece "hangi bilgiler geldi" bilgisi.

### ② Handler — "Nasıl yapılır?" (asıl iş burada)

```
Features/Cities/Commands/CreateCity/CreateCityCommandHandler.cs (özetlenmiş)

public class CreateCityCommandHandler : IRequestHandler<CreateCityCommand, long>
{
    private readonly ICityWriteRepository _cityWriteRepository;

    public async Task<long> Handle(CreateCityCommand request, CancellationToken ct)
    {
        var city = new City { CityName = request.CityName };

        await _cityWriteRepository.AddAsync(city, ct);
        await _cityWriteRepository.SaveAsync(ct); // DB'ye gerçekten yaz

        return city.Id;
    }
}
```

`IRequestHandler<CreateCityCommand, long>` imzası MediatR'a şunu söyler: "Biri sana bir `CreateCityCommand` gönderirse, beni çağır." Handler'ın tek görevi budur — belirli bir komutu işlemek. Constructor'a hangi bağımlılıklar (repository gibi) verilirse onları kullanabilir; ihtiyaç duymadığı hiçbir şeyi (HTTP, JSON, controller...) bilmez.

### ③ Validator — "Geçerli mi?" (otomatik çalışır, kimse elle çağırmaz)

```
Features/Cities/Commands/CreateCity/CreateCityCommandValidator.cs

public class CreateCityCommandValidator : AbstractValidator<CreateCityCommand>
{
    public CreateCityCommandValidator()
    {
        RuleFor(x => x.CityName)
            .NotEmpty().WithMessage("Şehir adı boş olamaz.")
            .MaxLength(100);
    }
}
```

Bu sınıf **hiçbir yerde elle çağırılmıyor**. FluentValidation kütüphanesinin kuralı: her `AbstractValidator<T>` otomatik keşfedilir ve `T` tipi bir komut MediatR'a her gönderildiğinde önce bu kural çalışır. Bunu Bölüm 2.5'te (Pipeline) detaylı göreceğiz.

## 2.4 MediatR nasıl devreye giriyor — DependencyInjection.cs

Peki MediatR, hangi komutun hangi Handler'a ait olduğunu **nereden** biliyor? Elle bir eşleştirme listesi yazmıyoruz — bunun yerine uygulama açılırken tüm dosyalar **otomatik taranıyor**:

```
Core/CineSeat.Application/DependencyInjection.cs

public static class DependencyInjection
{
    public static IServiceCollection AddApplicationServices(this IServiceCollection services)
    {
        var assembly = Assembly.GetExecutingAssembly();

        // Tüm IRequestHandler<,> implementasyonlarını otomatik bulur ve kaydeder.
        services.AddMediatR(cfg => cfg.RegisterServicesFromAssembly(assembly));

        // Tüm AbstractValidator<T> implementasyonlarını otomatik bulur ve kaydeder.
        services.AddValidatorsFromAssembly(assembly);

        // Pipeline halkaları - istek yukarıdan aşağıya sırayla geçer (bkz. 2.5).
        services.AddTransient(typeof(IPipelineBehavior<,>), typeof(LoggingBehavior<,>));
        services.AddTransient(typeof(IPipelineBehavior<,>), typeof(ValidationBehavior<,>));

        return services;
    }
}
```

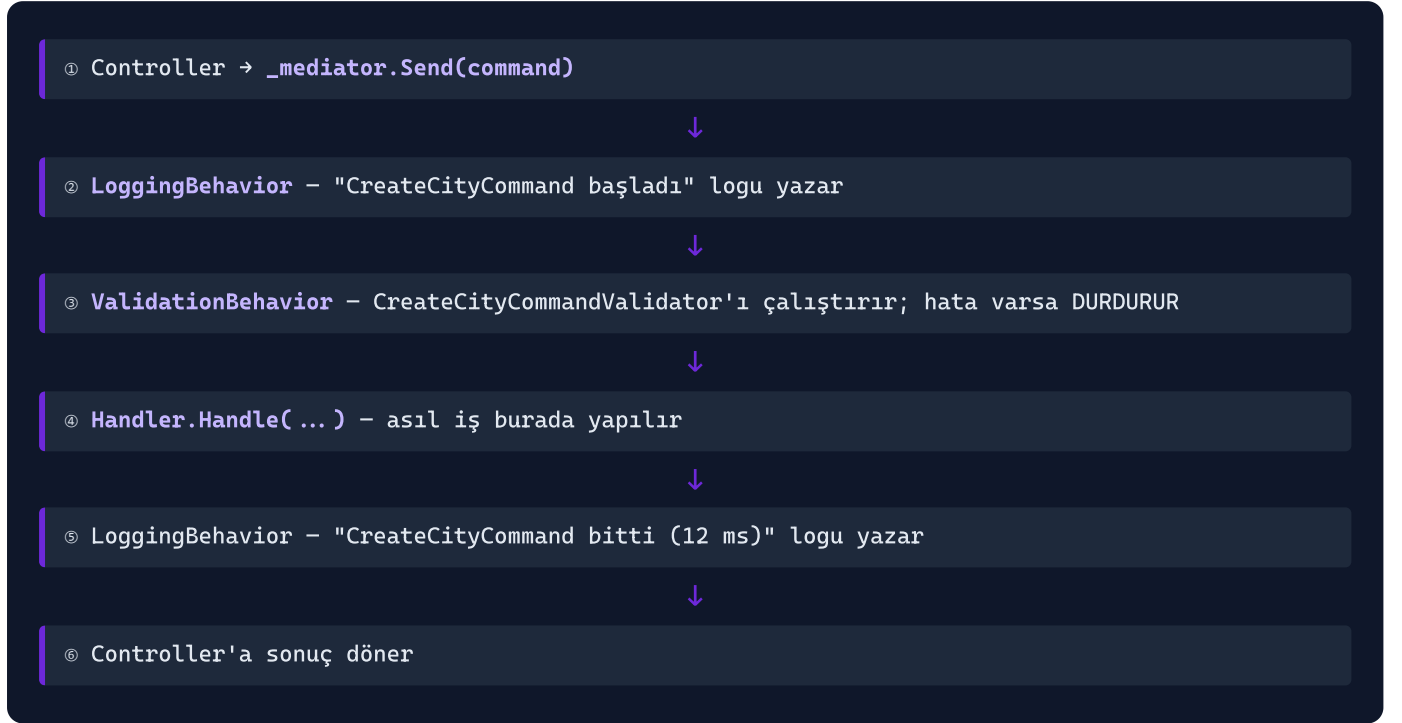
**Satır satır:**

- `Assembly.GetExecutingAssembly()` → "bu projedeki (Application.dll) tüm sınıfları getir"
- `cfg.RegisterServicesFromAssembly(assembly)` → MediatR o dosyaların içinde `IRequestHandler<,>` arayüzünü uygulayan **her** sınıfı bulur ve kaydeder. Bu satır sayesinde **yeni bir Handler eklediğinizde bu dosyaya asla dokunmazsınız** — otomatik bulunur.
- `AddValidatorsFromAssembly(assembly)` → aynı mantık, ama `AbstractValidator<T>` sınıfları için.

Bu tek dosya sayesinde `Program.cs` 'te sadece `builder.Services.AddApplicationServices();` yazmak yeterli — 90'dan fazla Handler'ın hiçbiri elle kaydedilmiyor.

## 2.5 Pipeline Behavior — isteğin geçtiği "borular"

Bir komut, doğrudan Handler'a gitmez. Önce birkaç "kontrol istasyonundan" geçer. Buna **Pipeline Behavior** denir. Bizde 2 tane var, sırayla çalışırlar:



**Benzetme:** Havaalanı güvenliği gibi düşünün. Uçağa (Handler) binmeden önce herkes aynı 2 kontrolden geçer: **check-in** (loglama — "bu yolcu geldi" kaydı) ve **bagaj taraması** (doğrulama — kurallara uymuyorsa *uçağa hiç binemez*). Her yeni yolcu (her yeni Command/Query) için bu kontrolleri yeniden yazmıyorsunuz — sistemin kendisi otomatik uyguluyor.

### ValidationBehavior — kod ve satır satır açıklama

Common/Behaviors/ValidationBehavior.cs

```

public class ValidationBehavior<TRequest, TResponse> : IPipelineBehavior<TRequest, TResponse>
{
    private readonly IEnumerable<IValidator<TRequest>> _validators;

    public async Task<TResponse> Handle(
        TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        // Bu istek için hiç validator kaydedilmemişse doğrudan handler'a geç.
        if (!_validators.Any()) return await next();

        var context = new ValidationContext<TRequest>(request);
        var results = await Task.WhenAll(
            _validators.Select(v => v.ValidateAsync(context, ct)));

        var failures = results.SelectMany(r => r.Errors).ToList();

        if (failures.Count > 0)
            throw new ValidationException(failures); // Handler HİÇ çağrılmaz!

        return await next(); // her şey temizse sıradaki halkaya (ya da Handler'a) geç
    }
}

```

**En kritik satır:** `RequestHandlerDelegate<TResponse> next` parametresi "zincirdeki bir sonraki adım"ı temsil eder. `next()` çağrılmazsa, zincir orada **durur** — Handler asla çalışmaz. `ValidationBehavior` tam olarak bunu yapıyor: hata varsa `next()` 'i hiç çağırmadan exception fırlatıyor. Böylece **hiçbir Handler'ın içinde tek satır doğrulama kodu yok** — hepsi bu tek dosyada merkezi olarak hallediliyor.

## LoggingBehavior — daha basit bir örnek

```

Common/Behaviors/LoggingBehavior.cs

public class LoggingBehavior<TRequest, TResponse> : IPipelineBehavior<TRequest, TResponse>
{
    public async Task<TResponse> Handle(
        TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        var requestName = typeof(TRequest).Name;
        var stopwatch = Stopwatch.StartNew();

        _logger.LogInformation("[CQRS] {RequestName} başladı", requestName);
        var response = await next(); // ← Handler (ya da sıradaki halka) burada çalışır
        _logger.LogInformation("[CQRS] {RequestName} bitti ({ms} ms)", requestName, stopwatch.ElapsedMilliseconds);

        return response;
    }
}

```

Bu, "cross-cutting concern" (her yeri ilgilendiren ama iş mantığıyla ilgisi olmayan bir kaygı) kavramının en net örneği. Süre ölçme kodu **tek bir yerde** yazıldı, ama **tüm** Command/Query'ler için otomatik çalışıyor — City'den Reservation'a kadar.

## 2.6 Bir isteğin A'dan Z'ye tam yolculuğu

Şimdi hepsini birleştirelim. Bir kullanıcı Postman/tarayıcıdan `POST /api/cities` isteği + `{"cityName": "İzmir"}` gönderdiğinde tam olarak neler oluyor:



1. HTTP isteği → `CitiesController.Create(command)` – ASP.NET, JSON gövdesini otomatik `CreateCityCommand` nesnesine çevirir (*model binding*)
- ↓
2. Controller: `await _mediator.Send(command)` – işi MediatR'a devreder
- ↓
3. `LoggingBehavior` devreye girer, "başladı" logu basar, `next()` çağırır
- ↓
4. `ValidationBehavior` devreye girer, `CreateCityCommandValidator`'ı bulur ve çalıştırır → `CityName` boş değil, geçerli → `next()` çağırır
- ↓
5. `CreateCityCommandHandler.Handle(...)` çalışır: yeni bir `City` nesnesi oluşturur, `ICityWriteRepository` üzerinden veritabanına ekler
- ↓
6. Repository, EF Core aracılığıyla PostgreSQL'e gerçek bir `INSERT INTO cities ...` SQL'i gönderir
- ↓
7. Yeni şehrin `Id`'si Handler'a, oradan `LoggingBehavior`'ın "bitti" logunu tetikleyerek Controller'a, oradan HTTP yanıtı olarak kullanıcıya döner

Bu yolculukta **Controller hiçbir zaman EF Core'u görmedi, Handler hiçbir zaman HTTP'yi görmedi, Validator hiçbir zaman veritabanını görmedi**. Her parça sadece kendi işini biliyor — bu, CQRS + MediatR'ın asıl kazandırdığı şey.

## 2.7 Repository Pattern — Application neden EF Core'u tanımıyor

Bölüm 1.2'de "Application, Domain dışında hiçbir şeyi tanımaz" demiştik. Peki Handler veritabanına nasıl erişiyor madem EF Core'u (Entity Framework — veritabanı kütüphanesi) tanımıyor? Cevap: **Repository** adında bir "aracı arayüz" üzerinden.

```
Core/CineSeat.Application/Repositories/IRepository.cs

/// Repository ailesinin ortak kökü. Bilinçli olarak ÜYESİZ bir marker interface'tir.
/// Eskiden burada DbSet<T> Table vardı; bu, Application katmanını
/// Microsoft.EntityFrameworkCore'a bağımlı kılıyordu.
public interface IRepository<T> where T : BaseEntity { }
```

Application katmanında sadece **arayüzler** (interface) var: `IReadRepository<T>`, `IWriteRepository<T>`. Bunların gerçek implementasyonu (EF Core kullanan kısım) `CineSeat.Persistence` projesinde yaşıyor:

```
Infrastructure/CineSeat.Persistence/Repositories/WriteRepository.cs (özet)
```

```
public class WriteRepository<T> : IWriteRepository<T> where T : BaseEntity
{
    private readonly ApplicationDbContext _context; // EF Core burada, Application'da DEĞİL

    protected DbSet<T> Table => _context.Set<T>();

    public async Task<bool> AddAsync(T model, CancellationToken ct)
    {
        var entry = await Table.AddAsync(model, ct);
        return entry.State == EntityState.Added;
    }
    // ...
}
```

Program açılırken ( `Persistence/ServiceRegistration.cs` ), "birisi `ICityWriteRepository` isterse, ona `CityWriteRepository` 'nin bir kopyasını ver" diye kaydedilir. Handler, arayüzü kullanır ama gerçekte hangi sınıfın çalıştığını **bilmesine gerek yoktur**.

## Okuma tarafında ekstra bir katman: IAsyncQueryExecutor

Bir de küçük ama zekice bir detay var. EF Core'un `ToListAsync()` , `CountAsync()` gibi metotları da Microsoft.EntityFrameworkCore paketine ait — Handler bunları doğrudan çağırırsa yine Application, EF'e bağımlı olurdu. Çözüm:

```
Common/Interfaces/IAsyncQueryExecutor.cs

public interface IAsyncQueryExecutor
{
    Task<List<T>> ToListAsync<T>(IQueryable<T> query, CancellationToken ct);
    Task<int> CountAsync<T>(IQueryable<T> query, CancellationToken ct);
    Task<bool> AnyAsync<T>(IQueryable<T> query, CancellationToken ct);
}
```

Bir Query Handler şöyle görünür:

```
var query = _cityReadRepository.GetAll(tracking: false)
    .Select(c => new CityDto { Id = c.Id, CityName = c.CityName });

return await _queryExecutor.ToListAsync(query, ct); // EF'i ÇAĞIRAN, ama EF'i BİLMEYEN Handler
```

`IQueryable<T>` ve LINQ ( `.Select` , `.Where` ) aslında .NET'in kendi kütüphanesine ait, EF Core'a değil — o yüzden Application'da serbestçe kullanılabilir. Ama sorguyu **gerçekten çalıştırıp** sonucu getirmek ( `ToListAsync` ) EF'e özel — bu da `IAsyncQueryExecutor` 'a devredilir. Gerçek implementasyonu ( `EfAsyncQueryExecutor` ) yine `Persistence` 'ta.

## 2.8 Hata yönetimi — tek try/catch mantığı

Bir Handler hata bulursa (örneğin "bu film yok") ne yapar? **Hiçbir Handler kendi içinde `try/catch` yazmaz**. Bunun yerine anlamlı bir istisna (exception) fırlatır:

```
if (movie is null)
    throw new NotFoundException("Film", request.MovieId);
```

Bu istisna, tüm zinciri (Handler → Behavior'lar → Controller) "delip geçer" ve en dışta, uygulamadaki **TEK** try/catch bloğunda yakalanır:

```
Presentation/CineSeat.WebAPI/Middleware/ExceptionHandlingMiddleware.cs
```

```
public async Task InvokeAsync(HttpContext context)
{
    try { await _next(context); } // tüm istek zincirini çalıştır
    catch (ValidationException ex) { // → 400 Bad Request }
    catch (NotFoundException ex) { // → 404 Not Found }
    catch (UnauthorizedException ex) { // → 401 Unauthorized }
    catch (ConflictException ex) { // → 409 Conflict }
    catch (Exception ex) { // → 500 (beklenmeyen her şey) }
}
```

| İstisna tipi                       | HTTP kodu | Ne zaman fırlatılır                                                 |
|------------------------------------|-----------|---------------------------------------------------------------------|
| <code>ValidationException</code>   | 400       | ValidationBehavior'dan otomatik (Bölüm 2.5)                         |
| <code>NotFoundException</code>     | 404       | İstenen kayıt yok (film, kullanıcı, rezervasyon...)                 |
| <code>UnauthorizedException</code> | 401       | Giriş yapılmamış / token geçersiz                                   |
| <code>ConflictException</code>     | 409       | Kural ihlali (aynı e-posta, dolu koltuk, aktif olmayan kampanya...) |

Bu sayede **90'dan fazla Handler'ın hiçbirinde** HTTP durum kodu bilgisi yok — her biri sadece "bu bir NotFound" der, hangi sayının (404) döneceğine tek bir yerde karar verilir.

## Bölüm 3 — Diğer Backend Eklemeleri

CQRS/MediatR temelini öğrendiğinize göre, artık üstüne inşa edilen her özelliği daha hızlı anlayabilirsiniz — çünkü hepsi Bölüm 2'deki aynı kalıbı (Command/Query + Handler + Validator) kullanıyor.

### 3.1 Kimlik doğrulama (Auth) ve JWT

**JWT (JSON Web Token)** — kullanıcı giriş yaptığında sunucunun ona verdiği "imzalı kimlik kartı". Kart, kullanıcının kim olduğunu (id, kullanıcı adı, rolü) şifresiz ama **değiştirilemez** biçimde taşır. Her istekte bu kart **Authorization** header'ında gönderilir; sunucu imzayı doğrular, kartın içeriğine güvenir — her seferinde veritabanına "bu gerçekten var mı" diye sormaz.

#### Register (Kayıt) akışı

Features/Auth/Commands/Register/RegisterCommandHandler.cs (özet)

```
public async Task<AuthResultDto> Handle(RegisterCommand request, Cancellation token ct)
{
    // 1) E-posta/kullanıcı adı zaten alınmış mı?
    if (emailTaken) throw new ConflictException("Bu e-posta adresi zaten kayıtlı.");

    // 2) Varsayılan "User" rolünü bul (Admin ASLA istemciden gelmez!)
    var role = await ... FirstOrDefaultAsync(r => r.Name == RoleNames.User, ct);

    // 3) Şifreyi HASH'le (düz metin asla saklanmaz - bkz. aşağıdaki kutu)
    var (hash, salt) = _passwordHasher.Hash(request.Password);

    var user = new User { ... , PasswordHash = hash, PasswordSalt = salt, RoleId = role.Id };
    await _userWrite.AddAsync(user, ct);
    await _userWrite.SaveAsync(ct);

    // 4) Giriş kartını (JWT) üret ve kullanıcıya ver
    var token = _tokenService.CreateAccessToken(user.Id, user.Username, user.Email, role.Name);
    return new AuthResultDto { Token = token.Token, User = ... };
}
```

**Bilinçli güvenlik kararı:** **RegisterCommand** 'da **Role** alanı **yok**. Herkes zorunlu olarak **"User"** rolüyle başlar. Eğer istemci "ben Admin olayım" diyebilseydi, herkes kendini yönetici yapardı!

#### Şifre nasıl saklanır? (PBKDF2)

Şifreler veritabanında **asla düz metin** olarak durmaz. "Hash'leme" — şifreyi geri döndürülemez bir matematiksel işlemden geçirip sonucu saklama — kullanılır:

Infrastructure/CineSeat.Infrastructure/Security/Pbkdf2PasswordHasher.cs

```
public (string Hash, string Salt) Hash(string password)
{
    var salt = RandomNumberGenerator.GetBytes(SaltSize); // her kullanıcı için FARKLI rastgele değer
    var hash = Rfc2898DeriveBytes.Pbkdf2(password, salt, Iterations: 100_000, ...);
    return (Convert.ToBase64String(hash), Convert.ToBase64String(salt));
}
```

- Salt (tuz):** Her kullanıcı için rastgele üretilen ekstra veri. İki kullanıcı aynı şifreyi kullansa bile, salt farklı olduğu için hash'leri **tamamen farklı** çıkar. Bu, hazır "şifre-hash tabloları" ile saldırıyı imkansız kılar.

- **100.000 iterasyon:** Hash işlemi bilerek yavaş yapılır (100 bin kez tekrarlanır) — bu, bir saldırganın saniyede deneyebileceği şifre sayısını ciddi şekilde düşürür.
- **FixedTimeEquals :** Doğrulamada normal `==` yerine bu kullanılır çünkü `==` ilk farklı bayt'ta karşılaştırmayı durdurur — bu *süre farkından* saldırganı ipucu sızdırabilir ("zamanlama saldırısı"). Sabit süreli karşılaştırma bu riski kapatır.

## Login akışı — kasıtlı "belirsiz" hata mesajı

Features/Auth/Commands/Login/LoginCommandHandler.cs (kritik satır)

```
if (candidate is null || !_passwordHasher.Verify(request.Password, candidate.PasswordHash, candidate.Password))
{
    throw new UnauthorizedException("Kullanıcı adı/e-posta veya parola hatalı.");
}
```

**Neden aynı mesaj?** "Kullanıcı bulunamadı" ile "Parola yanlış" için *farklı* mesaj dönsaydik, bir saldırgan deneme-yanılma ile hangi e-postaların sistemde kayıtlı olduğunu öğrenebilirdi. Tek, belirsiz mesaj bu bilgi sızıntısını kapatır.

## JWT üretimi

Infrastructure/CineSeat.Infrastructure/Security/JwtTokenService.cs (özet)

```
var claims = new List<Claim>
{
    new(ClaimTypes.NameIdentifier, userId.ToString()), // "sen kimsin" → ICurrentUserService bunu okur
    new(ClaimTypes.Name, username),
    new(ClaimTypes.Role, roleName), // "hangi rolsün" → [Authorize(Roles= ...)] bunu okur
};
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
var token = new JwtSecurityToken(issuer: ..., claims: claims, expires: ..., signingCredentials: ...);
```

**Claim** = token'ın içine gömülen bir "etiket". Kullanıcı her istek attığında bu etiketler otomatik okunur — sunucu her seferinde veritabanına gidip "bu kullanıcının rolü ne" diye sormaz, token'ın imzasına güvenir.

**appsettings.json'da düz metin anahtar var:** `"Key": "cineseat-development-only-signing-key-change-me-in-production"` — bu geliştirme için sorun değil ama **prod'a çıkmadan önce** ortam değişkenine taşınmalı ve anahtar yenilenmeli. (Ömer'in notlarında da işaretlenmiş açık madde.)

## 3.2 Yetkilendirme (RBAC) — kim neyi yapabilir

**RBAC = Role-Based Access Control** (Rol Tabanlı Erişim Kontrolü). Her kullanıcının bir **Role** 'ü var ( **Admin** veya **User** ), her Controller endpoint'i gerekli rolü **[Authorize]** özniteliğiyle ilan ediyor:

```
[Authorize(Roles = RoleNames.Admin)] // sadece Admin girebilir
[HttpPost]
public async Task<IActionResult> Create([FromBody] CreateCityCommand command) { ... }
```

Rol adları elle yazılan metin ("Admin") yerine sabitlerden geliyor — yazım hatası riski sıfır:

Common/Constants/RoleNames.cs

```
public static class RoleNames
{
    public const string Admin = "Admin";
    public const string User = "User";
}
```

**Kural:** Okuma uçları (film listeleme, koltuk haritası...) herkese açık; yazma uçları (ekleme/güncelleme/silme) Admin'e kapalı. Örnek: `ShowtimesController` 'da `GetByMovie` 'nin üstünde `[Authorize]` yok ama `Create` 'in üstünde `[Authorize(Roles = RoleNames.Admin)]` var.

## "Şu an giriş yapan kim?" — `ICurrentUserService`

Bir Handler, "bu rezervasyonu yapan kullanıcı kim" bilgisine nasıl ulaşır? İstekten değil — **token'dan**:

```
Presentation/CineSeat.WebAPI/Services/CurrentUserService.cs

public long? UserId
{
    get
    {
        var raw = Principal?.FindFirstValue(ClaimTypes.NameIdentifier); // token'daki claim'i oku
        return long.TryParse(raw, out var id) ? id : null;
    }
}
```

Bir Handler, `ICurrentUserService.GetRequiredUserId()` çağırır — bu, Application katmanında tanımlı bir **arayüz**; gerçek implementasyonu (yukarıdaki, HttpContext kullanan) yine WebAPI'de. Aynı Repository mantığı burada da tekrar ediyor: Application, "HttpContext" diye bir şeyin var olduğunu bile bilmiyor.

## 3.3 IDOR güvenlik açıkları ve kapatılması

**IDOR (Insecure Direct Object Reference)** — bir kullanıcının, sadece bir id tahmin/deneyerek **başkasının** verisine erişebilmesi. Örnek: giriş yapmış herhangi biri `GET /api/reservations/17` yazarak *başkasının* rezervasyonunu görebiliyor olsaydı, bu bir IDOR açığı olurdu.

Proje ilk yazıldığında rezervasyon/koltuk kilitleme uçları `userId` 'yi **istekten** alıyordu — kod çalışıyordu ama güvenli değildi. Auth eklenince bu 7 nokta tarandı ve düzeltildi:

| Uç nokta                                     | Açık                                                     |
|----------------------------------------------|----------------------------------------------------------|
| <code>GET /Reservations/my</code>            | Herkes başkasının rezervasyon listesini görebiliyordu    |
| <code>POST /Reservations</code>              | Başkası adına rezervasyon yapılabiliyordu                |
| <code>GET /Reservations/{id}</code>          | id deneyen herkes alıcı adı/e-posta/tutar okuyabiliyordu |
| <code>DELETE /SeatLocks/{id}</code>          | Herkes başkasının koltuk kilidini açabiliyordu           |
| <code>GET /Tickets/{id}</code> ve benzerleri | Aynı IDOR deseni                                         |

**Çözüm deseni** — her yerde aynı: `userId` parametresini komuttan tamamen kaldır, `ICurrentUserService` 'ten oku, ve sahiplik kontrolü ekle:

```
Features/Reservations/Queries/GetReservationById/GetReservationByIdQueryHandler.cs
```

```
var userId = _currentUser.GetRequiredUserId();
var isAdmin = _currentUser.Role == RoleNames.Admin;

var reservation = await _reservationRead.GetByIdAsync(request.Id, ...);

// Sahiplik kontrolü ŞART: olmasaydı id deneyen herkes başkasının
// rezervasyonunu okuyabilirdi.
if (reservation is null || (reservation.UserId != userId && !isAdmin))
    throw new NotFoundException("Rezervasyon", request.Id); // 403 DEĞİL, 404!
```

**Neden 403 değil 404?** **403 Forbidden** demek "bu kayıt var ama senin değil" — bu bile bir bilgi sızıntısıdır (kaydın *var olduğunu* öğrenirsiniz). Yetkisiz erişimde kaydın **varlığını bile** sızdırmamak için **NotFoundException** (404) fırlatılıyor — saldırgan için "bu id hiç yok" ile "bu id var ama senin değil" ayırt edilemez hale geliyor.

### 3.4 Katalog modülü (Film, Tür, Kampanya)

Bölüm 2'de öğrendiğiniz kalıbın (Command/Query/Handler/Validator) uygulandığı en büyük modüllerden biri. Öne çıkan noktalar:

- **Movie:** **GetMovieById** başta tüm listeyi çekip bellekte filtreliyordu ( **GetMoviesQuery{PageSize=100}** + **FirstOrDefault** ) — yavaş ve yanlış bir desendi; gerçek, tek satırlık bir sorguya çevrildi.
- **Genre:** Filme atanmış bir tür silinmeye çalışılırsa, veritabanı foreign-key hatası (anlaşılmaz) yerine anlaşılır bir **409 Conflict** döner.
- **Campaign:** Rezervasyonlarda kullanılmış bir kampanya **silinemez** (geçmiş kayıtları bozardı) — bunun yerine **IsActive=false** yapılması önerilir.

### 3.5 Sosyal modül (Yorum, Favori, Profil)

#### Yorum eklerken ortalama puanın otomatik güncellenmesi

Bir kullanıcı film puanladığında, filmin **AvgScore** (ortalama puan) alanı **anında yeniden hesaplanır**:

```
Features/Comments/Commands/AddComment/AddCommentCommandHandler.cs (özet)

// Bir kullanıcı bir filme TEK yorum yazabilir – aksi halde puan ortalaması
// tekrar tekrar yorum yazılarak manipüle edilebilirdi.
var alreadyCommented = await _queryExecutor.AnyAsync(
    _read.GetWhere(c => c.MovieId == request.MovieId && c.UserId == userId, ...), ct);
if (alreadyCommented) throw new ConflictException("Bu filme zaten yorum yaptınız.");

await _write.AddAsync(comment, ct);
await _write.SaveAsync(ct);

// Yeni yorum kaydedildikten SONRA hesaplanıyor ki ortalamaya dahil olsun.
movie.AvgScore = await MovieScoreCalculator.CalculateAsync(_read, _queryExecutor, request.MovieId, ct);
_movieWrite.Update(movie);
await _movieWrite.SaveAsync(ct);
```

**MovieScoreCalculator**, tüm yorumları belleğe çekip ortalama almak yerine **tek bir SQL sorgusuyla** ( **GroupBy** + **Select** ) ile veritabanının kendisine ortalamaı hesaplatırarak çok daha verimli çalışır.

## Profil — bilinçli olarak değiştirilemeyen alanlar

`UpdateProfile` komutunda `Email`, `Username` ve `Role` **yoktur**. Kimlik alanları (email/username) değişimi ayrı bir doğrulama akışı gerektirir (örn. e-posta onayı); rol değişimi ise doğrudan yetki yükseltmesi anlamına gelir — profil güncelleme ekranından asla yapılmamalı.

### 3.6 Konum & Mekan modülü (Şehir → Koltuk)

Fiziksel hiyerarşiyi modeller: `City → District → Cinema → Hall → Seat`. Her seviye bir öncekine `FK` (foreign key) ile bağlı, ve her Create/Update Handler'ı üst kaydın gerçekten var olduğunu doğrular:

```
var district = await _districtRead.GetByIdAsync(request.DistrictId, ...);  
if (district is null) throw new NotFoundException("İlçe", request.DistrictId);
```

**Koltuk (Seat)** modülünde ilginç bir detay: `CreateSeatsCommand` tek tek koltuk eklemek yerine **toplu üretim** yapar — "5 sıra × 8 sütun" dersiniz, 40 koltuk otomatik oluşur. Zaten var olan (sıra, sütun) çiftlerini atlayacak şekilde yazıldığı için aynı komutu tekrar çalıştırmak **hataya yol açmaz**, sadece eksikleri tamamlar (idempotent).

### 3.7 Rezervasyon akışı — en karmaşık parça

Bu, projedeki en çok adımlı iş kuralı. Bir `CreateReservationCommand` geldiğinde Handler sırayla şunları yapar:

1. Giriş yapan kullanıcı gerçekten var mı? (token geçerli ama kullanıcı silinmiş olabilir)



2. Seans var mı? Seçilen her koltuk var mı ve o seansın salonuna mı ait?



3. İstekte aynı koltuk iki kez mi seçilmiş? (reddet)



4. Koltuklardan biri zaten (iptal edilmemiş) rezerve mi? (409 Conflict)



5. Koltuklardan biri BAŞKA bir kullanıcı tarafından kilitli mi? (409 Conflict)



6. Fiyat hesabı: `seans.BasePrice × bilet-tipi çarpanı` (Yetişkin×1.0, Öğrenci×0.75, Çocuk×0.5)



7. Kampanya varsa: aktif mi? sepet eşiğini (`MinCartTotal`) geçiyor mu? indirimi hesapla



8. Reservation + Ticket'lar TEK `SaveChanges` ile veritabanına yazılır



9. Bu koltuklara ait varsa eski `SeatLock` kayıtları AYNI işlemde silinir



## Koltuk kilitleme (SeatLock) — eşzamanlılık problemi

İki kullanıcı **aynı anda** aynı koltuğu seçerse ne olur? Veritabanı seviyesinde bir **unique index** (benzersizlik kısıtı) (**ShowtimeId, SeatId**) çifti üzerinde tanımlı — aynı seans+koltuk için ikinci bir kilit satırı eklenmeye çalışılırsa PostgreSQL bunu **otomatik reddeder**. Kod bunu şöyle yönetir:

```
var existingLock = await _lockRead.GetSingleAsync(sl => sl.ShowtimeId == ... && sl.SeatId == ... , tracking:
if (existingLock is null)
{
    // hiç kilit yok → yeni satır oluştur
}
else if (existingLock.LockExpiresAt < now)
{
    // eski kilidin süresi dolmuş → satırı DEVRAL (yeni satır eklemeye çalışma, unique index'e takılırdı)
}
else if (existingLock.UserId != request.UserId)
{
    throw new ConflictException("Bu koltuk şu anda başka bir kullanıcı tarafından kilitli.");
}
```

Bu, "önce kontrol et sonra ekle" (check-then-insert) yaklaşımının veritabanı kısıtıyla birleştirilmiş hali — yarış durumunun (race condition) son savunma hattı her zaman veritabanının kendisidir, kod sadece kullanıcıya *anlaşılır* bir hata mesajı vermek için önden kontrol eder.

## Kampanya indirimi hesabı

CreateReservationCommandHandler.cs – CalculateDiscountAsync

```
if (!campaign.IsActive) throw new ConflictException("Bu kampanya aktif değil.");
if (subtotal < campaign.MinCartTotal) throw new ConflictException("Sepet tutarı yetersiz.");

var raw = campaign.Type switch
{
    CampaignType.Percentage => Math.Round(subtotal * campaign.Value / 100m, 2),
    CampaignType.FixedAmount => campaign.Value,
    _ => 0m
};
return Math.Min(raw, subtotal); // İNDİRİM ASLA ARA TOPLAMI AŞAMAZ
```

**Math.Min(raw, subtotal)** neden şart? Sabit tutarlı (FixedAmount) bir kampanyada indirim değeri (örn. 200 TL), sepet tutarından (örn. 85 TL) büyük olabilir. Bu satır olmasaydı **Total = Subtotal - Discount** işlemi **negatif** bir tutara düşerdi — müşteriye para ödemeniz gerekirdi!

## 3.8 Sayfalama (Pagination)

Bir liste (filmler, rezervasyonlar...) büyüdükçe hepsini tek seferde döndürmek hem yavaş hem gereksiz.

**PagedResult<T>** bunun standart çözümü:

Common/Models/PagedResult.cs

```
public class PagedResult<T>
{
    public IReadOnlyList<T> Items { get; }
    public int TotalCount { get; }
    public int Page { get; }
    public int PageSize { get; }
    public int TotalPages => (int)Math.Ceiling(TotalCount / (double)PageSize);
}
```

Bir Query Handler'da kullanımı:

```
var totalCount = await _executor.CountAsync(baseQuery, ct); // toplam kayıt sayısı

var pageQuery = baseQuery
    .OrderBy(c => c.CityName)
    .Skip((request.PageNumber - 1) * request.PageSize) // önceki sayfaları atla
    .Take(request.PageSize) // sadece bu sayfa kadarını al
    .Select(c => new CityDto { ... });

var items = await _executor.ToListAsync(pageQuery, ct);
return new PagedResult<CityDto>(items, totalCount, request.PageNumber, request.PageSize);
```

Sonuç JSON'da şöyle görünür: `{"items":[ ... ], "totalCount":47, "page":1, "pageSize":20, "totalPages":3}` — istemci (frontend) hem o sayfadaki veriyi hem de "kaç sayfa daha var" bilgisini tek istekte alır.

### 3.9 Otomatik kurulum (Migrate + Seed)

Uygulama her başladığında ( `Program.cs` ) otomatik olarak 2 şey yapar:

```
using (var scope = app.Services.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    var passwordHasher = scope.ServiceProvider.GetRequiredService<IPasswordHasher>();
    await DbInitializer.SeedAsync(context, passwordHasher);
}
```

Infrastructure/CineSeat.Persistence/Data/DbInitializer.cs (özet)

```
public static async Task SeedAsync(ApplicationDbContext context, IPasswordHasher hasher)
{
    await context.Database.MigrateAsync(); // bekleyen tüm migration'ları uygula

    if (await context.Roles.AnyAsync()) return; // zaten seed edilmişse ATLA (idempotent)

    // ... 7 izin, Admin/User rolü, admin@cineseat.com hesabı oluşturulur
}
```

- **MigrateAsync()** : Veritabanı şemasını (tablolar, kolonlar) kodun beklediği en güncel haline otomatik getirir — taze bir bilgisayarda projeyi çalıştıran biri elle hiçbir SQL komutu yazmaz.
- **Idempotent seed:** `if (Roles.Any()) return;` satırı, bu fonksiyonu 100 kere çalıştırsanız bile veriyi **tekrar tekrar eklemes** — sadece veritabanı gerçekten boşsa çalışır. (Bölüm 4'te bu satırla ilgili bulduğumuz gerçek bir durumu okuyacaksınız.)

## Bölüm 4 — Şu Anki Durum: Sağlık Kontrolü

Bu bölüm, backend'i gerçekten çalıştırıp test ederek yapılan denetimin özeti.

### Derleme (Build)

5 proje de hatasız derleniyor: **0 Hata** (76 uyarı — hepsi entity'lerdeki nullable alan uyarıları, işlevi etkilemiyor, yeni de değil).

### Uçtan uca test edilen akışlar (gerçek HTTP istekleriyle)

| Akış                                                           | Sonuç                                    |
|----------------------------------------------------------------|------------------------------------------|
| Kayıt olma, giriş yapma, yanlış şifre, mükerrer e-posta        | ✓ Hepsi doğru kodlarla (201/200/401/409) |
| Token'sız korumalı endpoint'e erişim                           | ✓ 401                                    |
| Normal kullanıcının Admin-only endpoint'e yazma denemesi       | ✓ 403                                    |
| Tür (Genre) ekleme, mükerrer tür                               | ✓ 201, tekrar → 409                      |
| Film + tür atama, yorum ekleme + AvgScore hesabı               | ✓ 5★+3★ → ortalama 4.00 doğru            |
| Aynı kullanıcı aynı filme ikinci yorum                         | ✓ 409 engellendi                         |
| Favori ekleme ve listeleme                                     | ✓                                        |
| Şehir → İlçe → Sinema → Salon → Koltuk → Seans zinciri         | ✓                                        |
| Kampanyalı rezervasyon (100 TL – %15 = 85 TL)                  | ✓ İndirim doğru hesaplandı               |
| Kampanya sepet eşiği altı                                      | ✓ 409 ile reddedildi                     |
| IDOR: başkasının rezervasyonunu okuma denemesi                 | ✓ 404 (sahibi/admin → 200)               |
| Anonim kullanıcı yazma denemeleri (City/Showtime/Reservations) | ✓ Hepsi 401                              |

### Bulunan ve düzeltilen 2 gerçek konu

#### Düzeltildi 1) Paylaşılan geliştirme veritabanı, admin hesabından yoksundu

Ömer'in `DbInitializer`'i `if (Roles.Any()) return;` ile idempotent çalışıyor — **doğru bir tasarım**. Ama bizim paylaşılan yerel PostgreSQL veritabanımızda `Roles` tablosu, bu kod yazılmadan **önceki** test oturumlarımdan zaten dolu geldiği için, bu koruma satırı admin hesabının ve izinlerin hiç oluşturulmasını engelledi. **Bu bir kod hatası değil** — taze bir veritabanında sorunsuz çalışır. Veriyi silmeden (register akışıyla doğru şifre hash'i üreterek + eksik izin/rol bağlarını SQL ile ekleyerek) düzelttim; `admin@cineseat.com / Admin123!` artık çalışıyor.

#### Bilgi 2) API yanıt biçiminde küçük bir tutarsızlık

"Kayıt oluştur" uçlarının **3'ü** (Genre, Movie, Campaign — Ömer'in yazdığı en yeni kod) `201 Created` + `{"id": 5}` döner; **diğer 12'si** (City, District, Cinema, Hall, Technology, HallTech, Showtime, Seat, SeatLock, Reservation, Comment, Favorite — çoğu daha önceki fazlardan) `200 OK` + çıplak sayı döner. Hiçbir işlevi **bozmuyor** — her uç

kendi içinde tutarlı çalışıyor — ama bir frontend geliştirici hangi uç neyi döndüğünü ezbere bilmek zorunda kalır. Küçük, isteğe bağlı bir iyileştirme fırsatı.

## Genel değerlendirme

Backend şu anda **sağlıklı ve production'a yakın** bir durumda: Auth, RBAC, IDOR korumaları, kampanya/rezervasyon iş kuralları, sayfalama, otomatik migrate+seed — hepsi gerçek HTTP istekleriyle doğrulandı. appsettings.json'daki JWT anahtarının prod öncesi değiştirilmesi ve yukarıdaki yanıt-biçimi tutarsızlığı, ekip için not edilmiş açık maddeler.